Version 2.0

January 2026

DATABASE

MODULE 6 - JOIN, SUB QUERY AND VIEW

Author:

Dr. Ir. Agus Eko Minarno, S.Kom., M.Kom., IPM.

Naufal Arkaan

Ismail Dwi Muh. Anugerah

INFORMATICS LABORATORY

University of Muhammadiyah Malang

TABLE OF CONTENTS

INTRODUCTION

OBJECTIVES

1. To enable students to understand the principles and applications of JOIN, SUBQUERY, and VIEW in SQL using Oracle.
2. To provide students with a structured understanding of Common Table Expressions (CTE) and analytic functions in database queries.
3. To equip students with both theoretical and practical skills to implement efficient SQL queries for real-world scenarios.

MODULE TARGETS

1. Students are able to perform various SQL JOIN operations, including INNER, OUTER, CROSS, and NATURAL JOIN.
2. Students are able to write and implement SUBQUERIES and VIEWS for complex data retrieval.
3. Students are able to apply Common Table Expressions (CTE) and basic analytic functions such as RANK and ROW_NUMBER using Oracle SQL.

PREPARATION

1. Muslim students may recite the following prayer before starting the lesson

   رَضِيْتُ بِاللهِ رَبًّا وَبِالإِسْلَامِ دِيْنًا وَبِمُحَمَّدٍ نَبِيًّا وَرَسُوْلًا. رَبِّ زِدْنِيْ عِلْمًا وَارْزُقْنِيْ فَهْمًا

2. Laptop or personal computer
3. **Oracle 11g XE:**
   LINK_ORACLE 11G XE
   Oracle apex:
   LINK_ORACLE APEX
   Student Affairs Scheme Link (Table) in the material explanation
   LINK_IMAGE OF THE STUDENT AFFAIRS TABLE
4. Strong motivation and commitment to learning

KEYWORDS

SQL JOIN, Subquery, View, CTE, Analytic Functions, Oracle

MATERIAL

1. SQL Join

Sometimes we need to use data from more than one table to create a report. What we need to do is link one table to another, and only then can we access data from both tables. Suppose that in an academic database system, data is not stored in one large table, but is separated into several tables according to their functions. In the student scheme used in this practicum, the data is divided into:

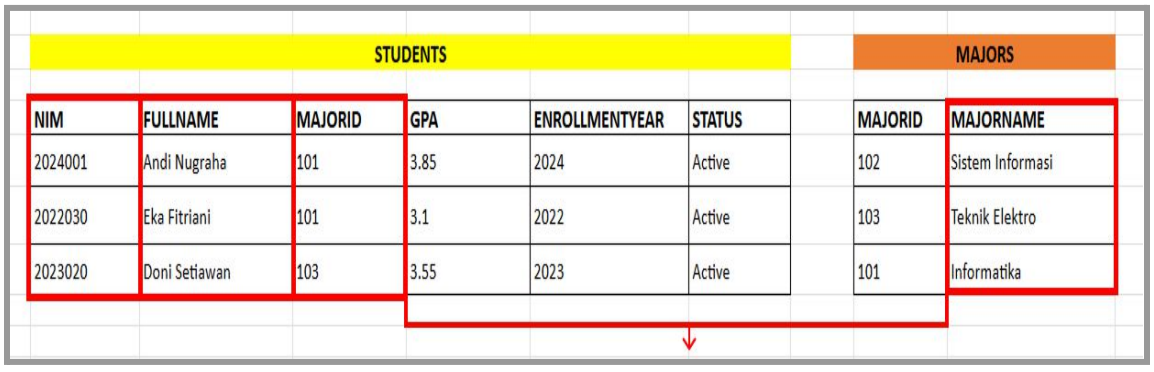
- **MAJORS** → store major data
- **STUDENTS** → store student data
- **LECTURERS** → store lecturer data
- **COURSES** → store course data

Each table is interconnected, but **not independent**. For example:

1. Student data **does not store the name of the department**, only the **MajorID**.
2. The name of the major is stored in the **MAJORS** table.
3. To display the student's name + department name, the two tables must be merged.

This merging process is called **`JOIN`**.

JOIN is an operation in SQL that is used **to combine rows of data from two or more tables based on related columns**, thereby producing more complete and meaningful information.
Example:



STUDENTS

NIM	FULLNAME	MAJORID	GPA	ENROLLMENTYEAR	STATUS
2024001	Andi Nugraha	101	3.85	2024	Active
2022030	Eka Fitriani	101	3.1	2022	Active
2023020	Doni Setiawan	103	3.55	2023	Active

MAJORS

MAJORID	MAJORNAME
102	Sistem Informasi
103	Teknik Elektro
101	Informatika

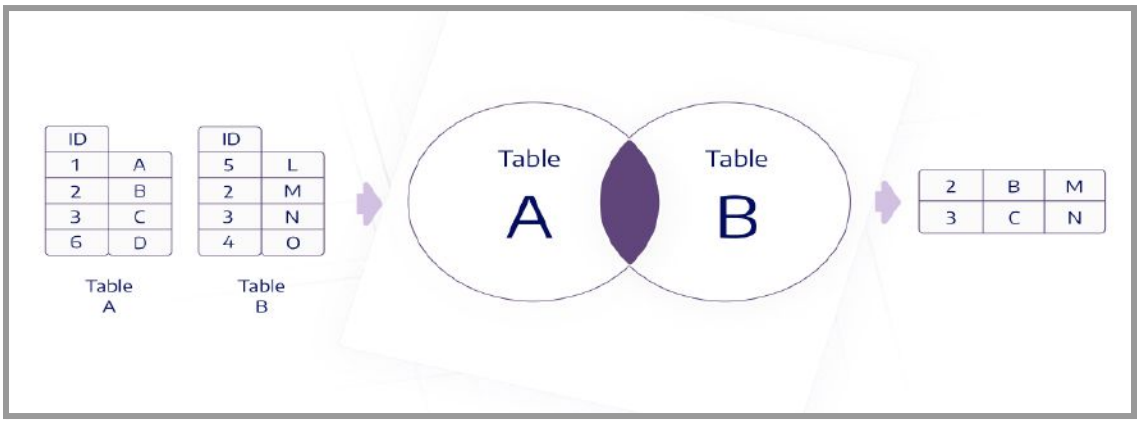
Join Result:

JOIN RESULT

NIM	FullName	MajorID	MajorName
2024001	Andi Nugraha	101	Informatika
2022030	Eka Fitriani	101	Informatika
2023020	Doni Setiawan	103	Teknik Elektro

In the example above, the **STUDENTS** table is related to the **MAJORS** table. The **MajorID** column in the **MAJORS** table acts as the **`Primary Key`**, while the **MajorID** column in the **STUDENTS** table acts as the **`Foreign Key`** that links student data with the major in which the student is enrolled.

1.1 Inner Join



INNER JOIN is a type of JOIN operation in SQL that is used to **combine data from two or more tables based on related columns**. **INNER JOIN** will only **display data rows that have matching values in both tables** according to the given **JOIN** conditions.

In other words, if a row in the first **table does not have a match** in the second table, **that row will not be displayed** in the query results.

In student schemes, **INNER JOIN** is often used to display:

- students and their majors,
- courses and their lecturers,
- or other related academic information.

Example:

Display **NIM**, **FullName**, **MajorID**, **MajorName** by combining the **STUDENTS** and **MAJORS** tables.

```
SELECT S.NIM, S.FullName, S.MajorID, M.MajorName
FROM STUDENTS S
INNER JOIN MAJORS M
ON S.MajorID = M.MajorID;
```

Output:

NIM	FULLNAME	MAJORID	MAJORNAME
2024001	Andi Nugraha	101	Informatika
2022050	Eka Fitriani	101	Informatika
2023020	Doni Setiawan	103	Teknik Elektro
2023015	Budi Cahyono	101	Informatika
2024002	Citra Dewi	102	Sistem Informasi

Explanation:

The result of the **INNER JOIN** command above is a combination of data from the **STUDENTS** and **MAJORS** tables based on the **MajorID** column. Each row of results displays student data that matches the **MajorID** with the major data. The student ID number and name information is taken from the **STUDENTS** table, while the major name is taken from the **MAJORS** table. Students who do not have a **MajorID** match in the **MAJORS** table will not be displayed in the **INNER JOIN** results.

1.2 Outer Join

OUTER JOIN is a type of **JOIN** in SQL that is used to combine data from two related tables, but still displays data from one or both tables even if no matching pair is found in the other table. Unlike **INNER JOIN**, which only displays matching data, **OUTER JOIN** allows data to appear with **NULL** values in corresponding columns that do not have a match.

In the student scheme, **OUTER JOIN** is useful for displaying all student data or all department data, including data that does not yet have a relationship with each other. **OUTER JOIN** consists of three types, namely **LEFT OUTER JOIN**, **RIGHT OUTER JOIN**, and **FULL OUTER JOIN**, each of which determines which side of the table remains displayed even if it does not have a data pair.

- **Left Outer Join**

`LEFT OUTER JOIN` is a type of **JOIN** operation in SQL that displays **all data from the left table** as well as data from the right table that matches the **JOIN** condition. If there is data in the left table that **does not have a match** in the right table, that data is still displayed with a **NULL** value in the right table column.

Example:
In this example, we will add data to the **LECTURERS** table with a **LecturerID** that does not exist in the **COURSES** table so that we can see the **NULL** value:

```
INSERT INTO LECTURERS (LecturerID, LecturerName, Email, MajorID)
VALUES (5004, 'Dr. Rudi Hartono', 'rudi.h@univ.ac.id', 102);
```

Query Left Outer Join:

```
SELECT L.LecturerName, C.CourseName
FROM LECTURERS L
LEFT OUTER JOIN COURSES C
ON L.LecturerID = C.LecturerID;
```

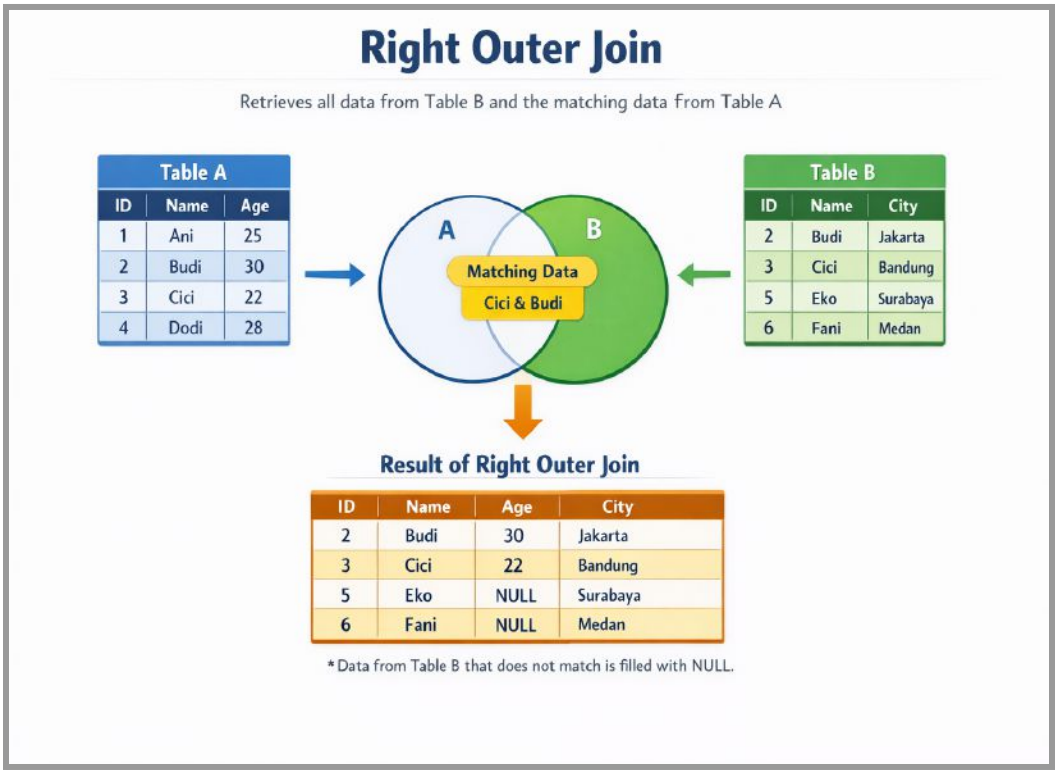
Output (See the bottom **COURSENAME** column with **NULL** values):

LECTURERNAME	COURSENAME
Prof. Cahyo Wibowo	Basis Data
Ir. Siti Aisyah	Analisis Sistem
Prof. Cahyo Wibowo	Jaringan Komputer
Dr. Budi Santoso	Pemrograman Dasar
Dr. Rudi Hartono	-

Explanation:

The **LEFT OUTER JOIN** above displays all lecturer data from the **LECTURERS** table as the main table. Lecturers who teach courses are displayed along with the course names, while lecturers who do not teach any courses are still displayed with a **NULL** value in the CourseName column. This condition shows that **LEFT OUTER JOIN** retains all data from the left table even if it does not have a data pair in the right table.

- **Right Outer Join**



`RIGHT OUTER JOIN` is a type of JOIN operation in SQL that **displays all data from the right table**, as well as data from the left table that matches the **`JOIN`** condition. If there is data in the right table that **does not have a match** in the left table, that data is still displayed with a **`NULL`** value in the left table column.

Example:
Add 1 Course WITHOUT a Lecturer, so we can see the difference between a right outer join.

```
INSERT INTO COURSES (CourseCode, CourseName, Credits, LecturerID)
VALUES ('IF202', 'Kecerdasan Buatan', 3, NULL);
```

Query Right Outer Join:

```
SELECT L.LecturerName, C.CourseName
FROM LECTURERS L
RIGHT OUTER JOIN COURSES C
ON L.LecturerID = C.LecturerID;
```

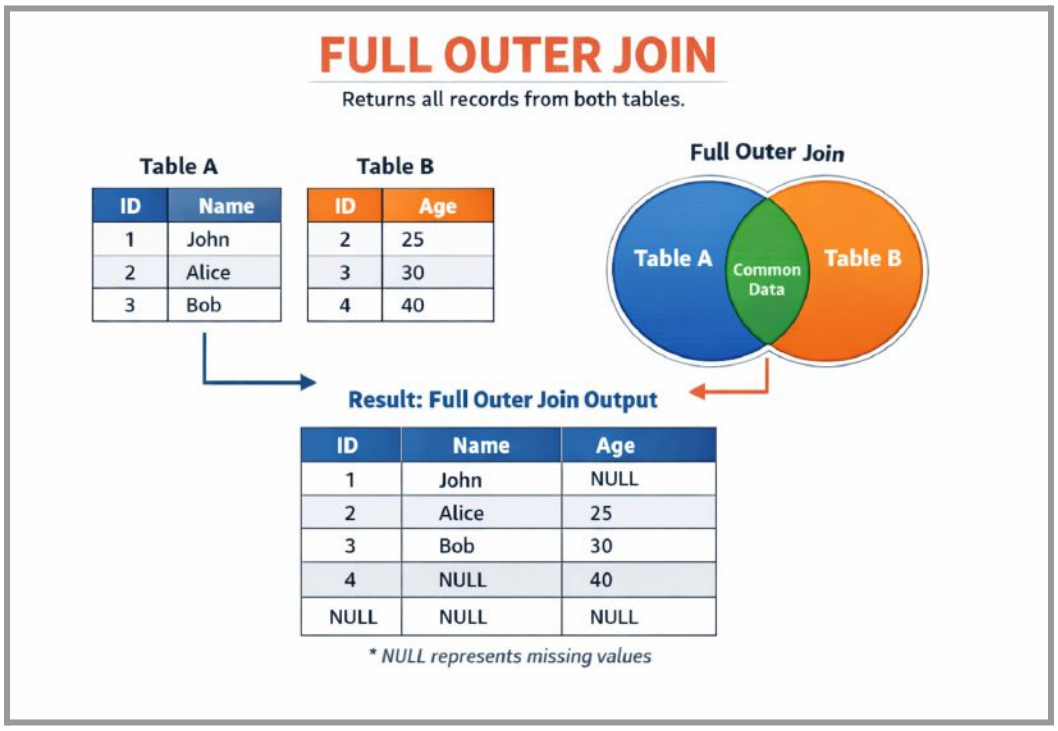
Output (See the bottom **LECTURERNAME** column **NULL**):

LECTURERNAME	COURSENAME
Prof. Cahyo Wibowo	Basis Data
Prof. Cahyo Wibowo	Jaringan Komputer
Dr. Budi Santoso	Pemrograman Dasar
Ir. Siti Aisyah	Analisis Sistem
-	Kecerdasan Buatan

Explanation:

The **`RIGHT OUTER JOIN`** above displays all course data from the **COURSES** table as the main table. Courses that already have a lecturer assigned will be displayed along with the lecturer's name, while courses that do not yet have a permanent lecturer will be displayed with a **NULL** value in the LecturerName column. This condition can be analogized as a list of all courses that have been compiled by the study program, even though not all of them have lecturers assigned to teach them.

- **Full Outer Join**



A `FULL OUTER JOIN` is a type of JOIN operation in SQL that **displays all data from both joined tables**, including data that has a match and data that does not have a match. If a row in one table does not have a match in the other table, the columns from the table that does not have a match will have a **NULL value**.

A **FULL OUTER JOIN** can be understood as a **combination of a LEFT OUTER JOIN and a RIGHT OUTER JOIN**, because it retains all data from both sides of the table.

Example:

Displaying the names of lecturers and course names, including:

- lecturers who have not yet taught the course, and
- courses that do not yet have a lecturer assigned to them.

```
SELECT L.LecturerName, C.CourseName
FROM LECTURERS L
FULL OUTER JOIN COURSES C
ON L.LecturerID = C.LecturerID;
```

Output:

LECTURERNAME	COURSENAME
Prof. Cahyo Wibowo	Basis Data
-	Kecerdasan Buatan
Ir. Siti Aisyah	Analisis Sistem
Prof. Cahyo Wibowo	Jaringan Komputer
Dr. Budi Santoso	Pemrograman Dasar
Dr. Rudi Hartono	-

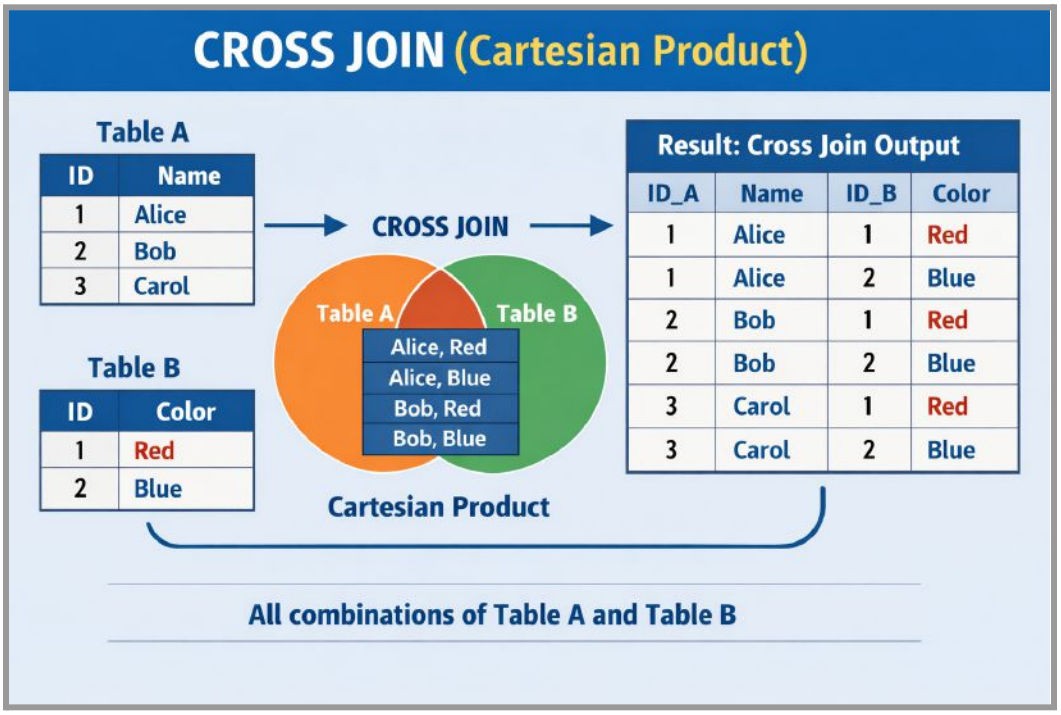
Description:

- Dr. Rudi Hartono is a lecturer who has not yet taught any courses.
- Artificial Intelligence/Kecerdasan Buatan is a course that does not yet have a lecturer.

Explanation:

The result of the **FULL OUTER JOIN** above displays all data from the **LECTURERS** table and all data from the **COURSES** table. Lecturers who teach courses will be displayed along with the course names, while lecturers who do not teach will still be displayed with a **NULL** value in the CourseName column. Conversely, courses that do not yet have a lecturer assigned to them will also be displayed with a **NULL** value in the LecturerName column. This condition can be analogized as a complete list of lecturers and a complete list of courses in a study program, where not all lecturers have been assigned a teaching schedule and not all courses have been assigned a lecturer.

1.3 Cross Join



CROSS JOIN is a **JOIN** operation in SQL that produces **all possible combinations of rows** from two joined tables. **CROSS JOIN does not use a join condition (ON)**, so each row in the first table will be paired with each row in the second table. The result of **CROSS JOIN** is often referred to as a **Cartesian product**.

Example:

Displaying **student names and department names**, combining all students with all departments.

```
SELECT S.FullName, M.MajorName
FROM STUDENTS S
CROSS JOIN MAJORS M;
```

Output:

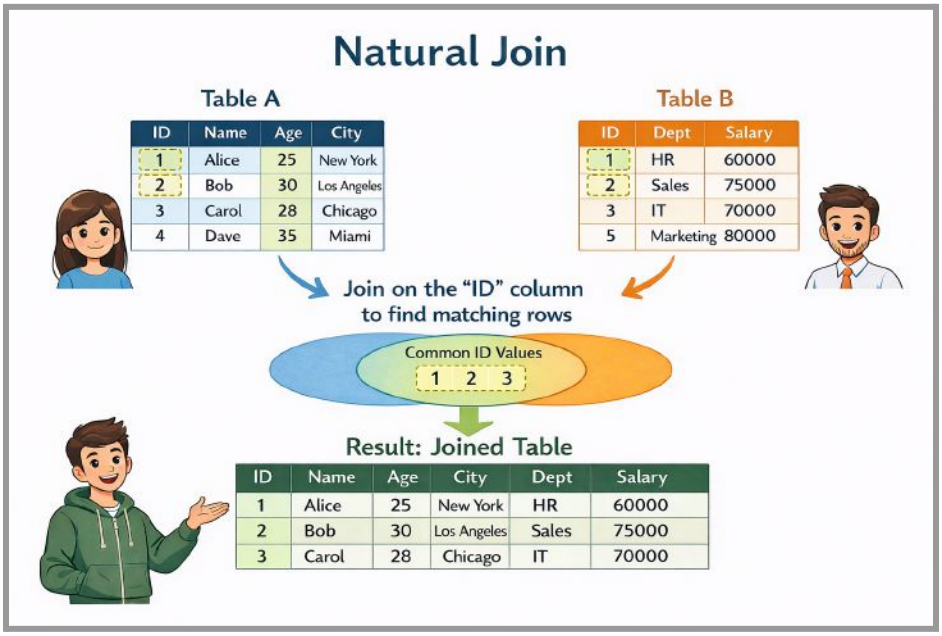
The actual output is a total of **15 rows** because the **STUDENTS** table has **5 rows x** the **MAJORS** table has **3 rows**, but Oracle APEX can **only display 10 rows**. To see the complete rows, please click **download** at the bottom of the table output.

FULLNAME	MAJORNAME
Andi Nugraha	Informatika
Andi Nugraha	Sistem Informasi
Andi Nugraha	Teknik Elektro
Eka Fitriani	Informatika
Eka Fitriani	Sistem Informasi
Eka Fitriani	Teknik Elektro
Doni Setiawan	Informatika
Doni Setiawan	Sistem Informasi
Doni Setiawan	Teknik Elektro
Budi Cahyono	Informatika
Budi Cahyono	Sistem Informasi
Budi Cahyono	Teknik Elektro
Citra Dewi	Informatika
Citra Dewi	Sistem Informasi
Citra Dewi	Teknik Elektro

Explanation:

The result of the **CROSS JOIN** above is a combination of all rows from the **STUDENTS** table with all rows from the **MAJORS** table without regard to the relationship between the tables. Each student will be paired with each major, even though in reality the student is only enrolled in one major. This condition can be analogized as a simulation of placing all students in all available majors, without considering the original majors of each student. Therefore, **CROSS JOIN** is rarely used to display real relational data, but is often used for simulation, system testing, or data combination purposes.

1.4 Natural Join



NATURAL JOIN is a JOIN operation in SQL that automatically combines two tables based on **columns that have the same name and data type**. In **NATURAL JOIN**, users **do not need to explicitly write JOIN conditions (ON)** because the database system will determine the connecting columns itself.

Example:

```
SELECT FullName, MajorName
FROM STUDENTS
NATURAL JOIN MAJORS;
```

Output:

FULLNAME	MAJORNAME
Andi Nugraha	Informatika
Eka Fitriani	Informatika
Doni Setiawan	Teknik Elektro
Budi Cahyono	Informatika
Citra Dewi	Sistem Informasi

Explanation:

The result of the NATURAL JOIN above is an automatic combination of the **STUDENTS** and **MAJORS** tables based on the **MajorID** column, which has the same name and data type. Each student will be displayed along with the corresponding major name without the need to write the JOIN condition

manually. This process can be analogized as a system that matches data based solely on column name similarities. Although practical, **NATURAL JOIN** must be used with caution as it can produce undesirable results if there is more than one column with the same name.

2. Subquery

A subquery is an SQL query written **within another SQL query**. Subqueries are used to generate a value or set of data that is then **used by the main query** as the basis for further processing.

In other words, subqueries function as auxiliary queries that help the main query make decisions or filter data. In student affairs, subqueries are often used to answer questions such as:

A. Students with the highest GPA
B. Students with above-average GPA
C. Students from specific majors
D. Courses taught by specific lecturers

Some important things to note when using subqueries:

1. **Subqueries are always executed first** before the main query.
2. Subqueries **must be enclosed in parentheses ( )**.
3. Subqueries can return:
    - a single value (single-row subquery)
    - more than one value (multi-row subquery)

✅ **Example of a correct subquery (Single-row Subquery)**:
Displaying students with the **highest GPA**.

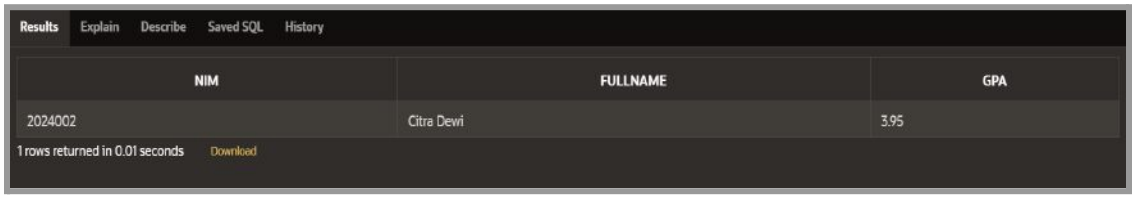
```
SELECT NIM, FullName, GPA
FROM STUDENTS
WHERE GPA = (
   SELECT MAX(GPA)
   FROM STUDENTS
);
```

Subquery:

```
SELECT MAX(GPA)
FROM STUDENTS
```

Produces a **single value**, so the use of the = operator is correct.

Output:



❌ **Examples of Incorrect Subqueries**:

```
SELECT NIM, FullName, GPA
FROM STUDENTS
WHERE GPA = (
   SELECT GPA
   FROM STUDENTS
);
```

INCORRECT, because the subquery returns **more than one row**, but the main query uses the = operator, which can only compare **one value**.

Explanation of Subqueries and Analogies:

A subquery can be likened to searching for supporting information before making a major decision, such as finding out the highest GPA before determining which students have that GPA.

2.1 Single-row Subquery

A single-row subquery is a subquery that **produces exactly one row (one value)** as its result. The value produced by this subquery is then used by the main query to perform comparisons using operators such as =, >, <, >=, or <=.

Single-row subqueries are commonly used when the desired result is **unique**, such as the maximum value, minimum value, or a specific aggregate calculation result.

Example:
Displaying students with the lowest GPA.

```
SELECT NIM, FullName, GPA
FROM STUDENTS
WHERE GPA = (
   SELECT MIN(GPA)
   FROM STUDENTS
);
```

Output:

Produces **the lowest GPA value**, so it is safe to use with the = operator.

Another example:
Displaying the **names of students with the highest GPA in the Informatics department**.

```
SELECT NIM, FullName, GPA
FROM STUDENTS
WHERE GPA = (
   SELECT MAX(GPA)
   FROM STUDENTS
   WHERE MajorID = (
      SELECT MajorID
      FROM MAJORS
      WHERE MajorName = 'Informatika'
   )
);
```

This query is used to display students with the highest GPA in the Computer Science department. The first subquery determines the **MajorID** for the Computer Science department from the **MAJORS** table. The next subquery calculates the highest GPA in that department. The highest GPA is then used by the main query to display data on students who have that GPA.

Output:

Results | Explain | Describe | Saved SQL | History

NIM	FULLNAME	GPA
2024001	Andi Nugraha	3.85

1 rows returned in 0.02 seconds Download

Explanation & Analogy:

A single-row subquery is used to obtain a **single reference value** that is then used by the main query in the data filtering process. The subquery is executed first to determine the value, then the main query uses the result to display the corresponding data. An analogy of this concept is like determining a specific benchmark, for example, the highest value, then using that benchmark to select data that meets the criteria.

2.2 Multi-row Subquery

A multi-row subquery is a subquery that **produces more than one row (multiple values)** as its result. Since the result is more than one value, the main query **cannot** use the **=** operator. Instead, special operators such as **`IN`**, **`ANY`**, or **`ALL`** are used.

Multi-row subqueries are suitable when search conditions **involve a lot of data**, such as students from several departments or courses from several lecturers.

Example:

Featuring students from the Computer Science and Information Systems departments.

```
SELECT NIM, FullName, MajorID
FROM STUDENTS
WHERE MajorID IN (
   SELECT MajorID
   FROM MAJORS
   WHERE MajorName IN ('Informatika', 'Sistem Informasi')
);
```

The subquery produces **more than one MajorID**, so the **`IN`** operator is used in the main query.

Output:

NIM	FULLNAME	MAJORID
2024001	Andi Nugraha	101
2022030	Eka Fitriani	101
2023015	Budi Cahyono	101
2024002	Citra Dewi	102

Another example:
Displaying courses taught by lecturers from the Informatics department.

```
SELECT CourseCode, CourseName
FROM COURSES
WHERE LecturerID IN (
   SELECT LecturerID
   FROM LECTURERS
   WHERE MajorID = (
      SELECT MajorID
      FROM MAJORS
      WHERE MajorName = 'Informatika'
   )
);
```

Brief outline:

- The deepest subquery searches for the **MajorID of the Informatics department.**
- The second subquery retrieves **all LecturerIDs** from that department.
- The main query displays the **courses taught by these lecturers**.

Since the subquery produces **more than one LecturerID**, the **`IN`** operator is used.

Output:

COURSECODE	COURSENAME
IF102	Basis Data
EL305	Jaringan Komputer
IF101	Pemrograman Dasar

Explanation & Analogy:

Multi-row subqueries are used when a single condition requires **more than one reference** value. The subquery is executed first to generate a list of values, then the main query matches the data with that list. An analogy for a multi-row subquery is like creating a list of criteria first, for example, several majors or several lecturers, then selecting the data that is included in that list.

3. View

A VIEW is a **virtual table** created from the results of an SQL query. A VIEW does not physically store data, but only stores the **query definition** that will be executed each time the VIEW is called. The data displayed by a `VIEW` always follows the latest data in the source table.

`VIEW` is used to simplify complex queries, improve data security, and make it easier for users to access specific information without having to know the table structure in detail.

3.1 Create View

`CREATE VIEW` is an SQL command used to **create a VIEW**, which is a virtual table derived from the results of a query. A VIEW stores the query definition, not physical data, so the data displayed by the VIEW always corresponds to the latest data in its source table.

Basic Structure of CREATE VIEW:

```
CREATE VIEW view_name AS
SELECT column1, column2, ...
FROM table_name
WHERE condition;
```

Description:

- **view_name**: name of the `VIEW` to be created
- **SELECT**: the query that forms the basis of the `VIEW`
- **`WHERE`**: conditions (optional)

Example View: Student Data and Majors

Create a VIEW to display **student ID numbers, student names, and department names**.

```
CREATE VIEW v_mahasiswa_jurusan AS
SELECT S.NIM, S.FullName, M.MajorName
FROM STUDENTS S
```

```
JOIN MAJORS M
ON S.MajorID = M.MajorID;
```

After the VIEW is created, the data can be displayed with the command:

```
SELECT * FROM v_mahasiswa_jurusan;
```

NIM	FULLNAME	MAJORNAME
2024001	Andi Nugraha	Informatika
2022050	Eka Fitriani	Informatika
2023020	Doni Setiawan	Teknik Elektro
2023015	Budi Cahyono	Informatika
2024002	Citra Dewi	Sistem Informasi

`VIEW` **v_mahasiswa_jurusan** will display student data along with their majors without the need to write **JOIN** repeatedly.

3.2 Replace View

`CREATE OR REPLACE VIEW` is an SQL command used to **change or update the definition of an existing VIEW** without having to delete the VIEW first. If the VIEW does not exist, this command will create a new VIEW. If the VIEW already exists, its definition will be replaced with the new one.

Basic Structure of CREATE OR REPLACE VIEW:

```
CREATE OR REPLACE VIEW view_name AS
SELECT column1, column2, ...
FROM table_name
WHERE condition;
```

Description:

- **view_name**: the name of the `VIEW` to be created or updated
- **SELECT**: a new query that becomes the definition of `VIEW`
- **WHERE**: condition (optional)

Example of CREATE OR REPLACE VIEW:
Example: Updating the Student and Department View
Suppose that the `VIEW` **v_mahasiswa_jurusan** already exists.
Now, the `VIEW` is updated by adding the **GPA** column.

```
CREATE OR REPLACE VIEW v_mahasiswa_jurusan AS
SELECT S.NIM, S.FullName, S.GPA, M.MajorName
FROM STUDENTS S
JOIN MAJORS M
ON S.MajorID = M.MajorID;
```

After being replaced, VIEW can be called again with:

```
SELECT * FROM v_mahasiswa_jurusan;
```

NIM	FULLNAME	GPA	MAJORNAME
2024001	Andi Nugraha	3.85	Informatika
2022030	Eka Fitriani	3.1	Informatika
2023020	Doni Setiawan	3.55	Teknik Elektro
2023015	Budi Cahyono	3.2	Informatika
2024002	Citra Dewi	3.95	Sistem Informasi

This command will display student data along with their **major and GPA** according to the latest VIEW definition.

3.3 Drop View

`DROP VIEW` is an SQL command used to **delete a VIEW** from a database. This command only deletes the **VIEW definition**, not the data in the source table. Once a VIEW is deleted, it cannot be used again unless it is recreated with the **`CREATE VIEW`** command.

Basic Structure of DROP VIEW:

```
DROP VIEW view_name;
```

3.4 Update View

`UPDATE VIEW` is an SQL command used to **update data through a VIEW**. Changes made to the VIEW actually **directly affect the data in the source table**, as long as the VIEW meets the requirements for an **updatable view** (for example, it comes from a single main table and does not use aggregate functions).

Basic Structure UPDATE VIEW:

```
UPDATE view_name
SET column_name = new_value
WHERE condition;
```

Description:

- **view_name**: the name of the VIEW to be created or updated
- **SELECT**: a new query that becomes the definition of VIEW
- **`WHERE`**: condition (optional)

Example of UPDATE VIEW:
Suppose there is the following VIEW:

```
CREATE VIEW v_data_mahasiswa AS
SELECT NIM, FullName, GPA, Status
FROM STUDENTS;
```

Example of Data Update via VIEW
Changing the GPA of a student with student ID number 2024001:

```
UPDATE v_data_mahasiswa
SET GPA = 3.90
WHERE NIM = '2024001';
```

This command will **change the GPA value in the STUDENTS table**, even though the UPDATE command is executed through VIEW.

4. Common Table Expressions (CTE)

Common Table Expressions (CTE) are an SQL feature used to **create temporary query results that can be called back in the main query**. CTEs are written using the **`WITH`** clause and aim to make queries neater, easier to read, and easier to understand, especially when queries have complex logic.

CTE is temporary and **only applies while the query is running**.

CTE Query Example (Student Scheme)

Displays students with a GPA above the average GPA of all students.

```
WITH RataIPK AS (
   SELECT AVG(GPA) AS AvgGPA
   FROM STUDENTS
)
SELECT NIM, FullName, GPA
FROM STUDENTS
WHERE GPA > (SELECT AvgGPA FROM RataIPK);
```

Explanation & Analogy:

In the example above, CTE is used to calculate the **average GPA** of students and store it temporarily under the name **RataIPK**. The value is then used by the main query to display students who have a GPA above average. CTE can be analogized as writing down important calculation results in a temporary note, then using that note as a reference in further decision making.

SUMMARY TABLE of MATERIALS

Query	How to Read Indonesian (Brief Analogy)
SELECT S.FullName, M.MajorName FROM STUDENTS S INNER JOIN MAJORS M ON S.MajorID = M.MajorID;	Ambil nama mahasiswa dan nama jurusannya, **hanya untuk mahasiswa yang jurusannya benar-benar ada dan cocok**.
SELECT L.LecturerName, C.CourseName FROM LECTURERS L LEFT JOIN COURSES C ON L.LecturerID = C.LecturerID;	Tampilkan semua dosen, **meskipun ada dosen yang belum mengajar mata kuliah**.
SELECT L.LecturerName, C.CourseName FROM LECTURERS L RIGHT JOIN COURSES C ON L.LecturerID = C.LecturerID;	Tampilkan semua mata kuliah, **meskipun ada mata kuliah yang belum punya dosen**.
SELECT L.LecturerName, C.CourseName FROM LECTURERS L FULL JOIN COURSES C ON L.LecturerID = C.LecturerID;	Tampilkan **semua dosen dan semua mata kuliah**, baik yang saling berhubungan maupun yang belum.
SELECT S.FullName, M.MajorName FROM STUDENTS S CROSS JOIN MAJORS M;	Pasangkan **setiap mahasiswa dengan semua jurusan**, tanpa peduli jurusan aslinya.
SELECT FullName, MajorName FROM STUDENTS NATURAL JOIN MAJORS;	Gabungkan mahasiswa dan jurusan **secara otomatis berdasarkan kolom yang namanya sama**.
SELECT NIM, FullName, GPA FROM STUDENTS WHERE GPA = (SELECT MIN(GPA) FROM STUDENTS);	Tampilkan mahasiswa yang **IPK-nya paling rendah**.
SELECT CourseCode, CourseName FROM COURSES WHERE Credits = (SELECT MAX(Credits) FROM COURSES);	Cari mata kuliah dengan **jumlah SKS paling besar**.
SELECT NIM, FullName FROM STUDENTS WHERE MajorID IN (SELECT MajorID FROM MAJORS WHERE MajorName IN ('Informatika','Sistem Informasi'));	Tampilkan mahasiswa yang **berasal dari jurusan Informatika atau Sistem Informasi**.
CREATE VIEW v_mahasiswa_jurusan AS SELECT S.NIM, S.FullName, M.MajorName FROM STUDENTS S JOIN MAJORS M ON S.MajorID = M.MajorID;	Buat **tabel virtual** yang menyimpan data mahasiswa beserta jurusannya agar tidak perlu menulis JOIN berulang.
UPDATE v_data_mahasiswa SET GPA = 3.90	Ubah IPK mahasiswa tertentu **melalui VIEW**,

WHERE NIM = '2024001';	tapi datanya tetap berubah di tabel asli.
WITH RataIPK AS (SELECT AVG(GPA) AS AvgGPA FROM STUDENTS) SELECT FullName, GPA FROM STUDENTS WHERE GPA > (SELECT AvgGPA FROM RataIPK);	Hitung dulu **rata-rata IPK**, lalu tampilkan mahasiswa yang IPK-nya **di atas rata-rata tersebut**.

PRACTICE ACTIVITY

In this practice activity, students are asked to create a new database schema with a theme of their choice based on their creativity. The schema must be logical and consist of at least two tables that are related to each other. After the schema and data have been created, students are required to apply several query concepts that they have learned in Module 6.

Instructions:

1. Create **a new database schema with a free theme** (e.g., online store, library, clinic, equipment rental, event system, etc.).
2. Ensure that the created schema has **relationships between tables** that enable the application of **JOIN** and **subqueries**.
3. Use the scheme you have created to construct the following queries:
    - **`LEFT JOIN`**, to display all data from the main table even if it does not have a match in another table.
    - **`Single-row Subquery`**, to display data with the highest or lowest values based on a criterion.
    - **`CREATE VIEW`**, to create a view from query results involving more than one table.
    - **`SELECT from VIEW`**, to display specific data from the view that has been created.
    - **`CTE (Common Table Expressions)`**, to calculate or summarize data as added value.
4. Run each query and include the output results obtained.
5. The results will be presented at the Query Dock session.

LEARNING RESOURCES AND TUTORIALS

LINK_W3SCHOOLS_SQL-JOIN **(Recommendations)**
LINK_YOUTUBE_SQL-JOINS **(Recommendations)**
LINK_DOCS-ORACLE

QUERY DOCK

Query Dock is a discussion and presentation session that aims to review and deepen the students' understanding of the material in Module 6. In this session, students will present the results of the **Practice Activity** that they have completed previously and actively discuss them with lab assistants and other students.

Query Dock Activity Flow:

1. **Presentation of Practice Activity Results**
   Students are asked to present:
   - The database schema created (themes and relationships between tables)
   - Queries that have been worked on include:
     - `LEFT JOIN`
     - Single-row Subquery
     - `CREATE VIEW` and `SELECT` from `VIEW`
     - CTE
   - The output result of each query
2. **Query Explanation**
   For each query presented, students are expected to be able to:
   - Explain the function and purpose of the query.
   - Convey the logic of the query in simple language.
   - Explain the reasons for using concepts (JOIN, subquery, VIEW, or CTE).
3. **Question and Answer Session Between students**
   Other students are welcome to:
   - Ask questions related to the query or schema presented
   - Provide input or opinions during the discussion session
4. **Questions from the Lab Assistant**
   If the students have understood the material and have no further questions, the lab assistant will ask additional questions related to Module 6, such as:
   - The Difference Between `LEFT JOIN` and `INNER JOIN`
   - The Difference Between Single-Row and Multi-Row Subqueries
   - The VIEW Function and Reasons for Using It
   - The Usefulness of CTE Compared to Subqueries

ASSIGNMENT (LIVE DEMO)

Live Demo is a live evaluation session that aims to test the students' understanding of Module 6 material independently and in real-time. In this session, students are required to be able to analyze table outputs and compile appropriate SQL queries without being given sample queries beforehand.

Live Demo Terms and Conditions:

1. Students are **not given SQL queries**, but only the **table output results** that must be analyzed.
2. Students are required to:
    - Understanding the structure of the given table
    - Write the correct SQL query so that it produces the same table output as given.
3. The time allowed for the Live Demo for each student is a maximum of **30 minutes**.

Note:

During the Live Demo session, the assessment focuses not only on the final query results, but also on the **thinking process, logic, and explanations** provided by the students in constructing queries based on the given table outputs.

PRACTICUM GRADING RUBRIC

Assessment Aspects	Points
Query Dock	**Total 10%**
Assignment (LIVE DEMO)	**Total 90%**
Question 1	5%
Question 2	10%
Question 3	25%
Question 4	25%
Question 5	25%

Important Notes on Practicum Implementation

During **Query Dock** week, students are encouraged to **ask questions** about module 6 material that they do not yet understand. This session not only tests syntax, but also helps students deepen their understanding of SQL concepts through discussions with assistants.

The **Query Dock score has a small percentage**, so the main focus of the assessment remains on **understanding the material** and the student's ability to explain the logic of the query.

For each assessment item on the rubric:

- **100 → Correct and confident answer / clear explanation**
- **50 → Partially correct answer / still uncertain**
- **0 → Unable to answer / does not follow instructions**

With this scheme, assessment is carried out objectively and consistently according to the level of student mastery.